DOCKER

Chuleta sencilla · comandos útiles para el examen

1. Las 3 ideas básicas

Imagen: plantilla con la que se crea un contenedor. **Contenedor:** imagen en ejecución. **Volumen:** datos que permanecen aunque borres el contenedor.

Imagen = plantilla · contenedor = programa funcionando · volumen = datos que permanecen

2. Comandos que más vas a usar

Comando	Para qué sirve
docker --version	Comprobar versión
docker pull nginx	Descargar imagen
docker images	Ver imágenes
docker run nginx	Crear y arrancar contenedor
docker ps / docker ps -a	Ver contenedores en marcha / todos
docker stop nombre	Parar contenedor
docker start nombre	Arrancar uno existente
docker rm nombre / docker rm -f nombre	Eliminar contenedor / forzar eliminación
docker rmi nginx	Eliminar imagen
docker logs nombre	Ver mensajes del contenedor
docker exec -it nombre sh	Entrar en su terminal

Ojo: `rm` elimina contenedores; `rmi` elimina imágenes.

3. Crear contenedores: parámetros importantes

```
docker run -d --name web -p 8080:80 nginx
```

Parte	Significado
run	Crea y arranca
-d	Segundo plano
--name web	Nombre del contenedor
-p 8080:80	Puerto del ordenador : puerto del contenedor
-it	Terminal interactiva
--rm	Borra el contenedor al terminar
-e CLAVE=valor	Variable de entorno
-v volumen:ruta	Monta almacenamiento
--network red	Conecta a una red

La web del ejemplo se abriría en **http://localhost:8080**.

4. Dockerfile: crear tu propia imagen

Un Dockerfile es un archivo de instrucciones para construir una imagen.

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

Instrucción	Qué hace
FROM	Elige imagen base
WORKDIR	Establece carpeta de trabajo
COPY	Copia archivos a la imagen
RUN	Ejecuta algo al construir
CMD	Comando predeterminado al arrancar

Construir y arrancar:

```
docker build -t mi-app:1.0 .
docker run --name prueba mi-app:1.0
```

Clave: RUN se ejecuta al construir; CMD se ejecuta al arrancar. EXPOSE documenta un puerto, pero no lo publica: para eso se usa -p.

5. Redes: conectar contenedores

```
docker network create mi-red
docker run -d --name servidor --network mi-red nginx:alpine
docker run -it --name cliente --network mi-red alpine:latest sh
```

Dentro del cliente:

```
ping -c 3 servidor
```

Los contenedores de una misma red creada por el usuario pueden comunicarse usando su nombre. Para comunicación interna no hace falta publicar puertos con -p.

```
docker network ls
```

6. Volúmenes: conservar datos

```
docker volume create mis-datos
docker run -d --name bd -v mis-datos:/var/lib/mysql mysql:8.0
docker volume ls
```

Si eliminas el contenedor, el volumen sigue existiendo. Si creas otro contenedor y montas el mismo volumen en la ruta correcta, puede recuperar los datos.

Volumen: datos persistentes (por ejemplo, una base de datos). **Bind mount:** conecta una carpeta concreta del ordenador con una carpeta del contenedor.

7. No confundas estos conceptos

Concepto	Recuerda
docker run	Crea y arranca
docker start	Arranca uno que ya existe
docker stop	Lo para
docker rm	Elimina un contenedor
docker rmi	Elimina una imagen
-p	Publica/conecta puertos
-v	Monta almacenamiento
--network	Conecta a una red
RUN / CMD	Construcción / arranque

8. Si algo falla, comprueba esto

1. ¿Docker Desktop está abierto y funcionando?
2. ¿Existe la imagen? `docker images`
3. ¿Está arrancado el contenedor? `docker ps -a`
4. ¿Qué error aparece? `docker logs nombre`
5. ¿El nombre ya está usado? Mira `docker ps -a`
6. ¿El puerto está ocupado? Prueba otro puerto del ordenador, por ejemplo `8081:80`.

Examen sin Internet

Las imágenes deben estar descargadas antes. `docker run` y `docker build` pueden funcionar sin Internet si las imágenes base y las dependencias necesarias ya están disponibles. Descargar imágenes o instalar paquetes desde Internet no funcionará sin conexión.

Consejo: empieza por dominar los comandos de las secciones 2 a 6. Son la base para reconocer y resolver la mayoría de tareas del test.